CENTRO UNIVERSITÁRIO FG — UNIFG

Ciência da Computação e Análise e Desenvolvimento de Sistemas · Matemática Computacional Aplicada (2026.2) · Prof. Ciro Chaves

Avaliação A3 — Projeto Prático Integrador

Motor de Detecção de Fraudes Financeiras e Lavagem de Capitais (FinTech Analytics)

VALOR TOTAL	MENTORIA / VALIDAÇÃO	ENTREGA E DEFESA	STACK OFICIAL
40,0 Pontos	**24/11/2026**	**01/12/2026**	**Python 3 + Pandas**

1. Contextualização e Cenário de Engenharia

Instituições financeiras digitais (FinTechs) processam milhares de transferências instantâneas por segundo. Em meio a operações legítimas, organizações criminosas utilizam redes de contas de passagem ("contas laranja"), compartilhamento ilícito de dispositivos físicos e esquemas de triangulação financeira para ocultar a origem de recursos e burlar limites regulatórios.

Neste projeto integrador, a sua equipe atuará como o núcleo de **Engenharia de Dados e Inteligência Antifraude** de uma FinTech submetida a uma auditoria do Banco Central. O objetivo do grupo é desenvolver e apresentar um pipeline computacional e matemático automatizado em Python capaz de auditar quatro camadas críticas de operação: a telemetria lógica de segurança dos dispositivos em baixo nível, a integridade relacional das bases cadastrais, o comportamento estatístico-probabilístico das transações e a topologia em grafos do fluxo monetário.

2. Semente Determinística e Geração dos Dados Exclusivos

Embora todas as equipes sigam rigorosamente a mesma arquitetura analítica, cada grupo trabalhará sobre um conjunto de dados exclusivo, contendo anomalias, valores e redes de fraude matematicamente únicos. A geração do cenário da equipe é controlada por uma **Semente Determinística (S_{eq})**, calculada pela soma aritmética simples dos números de Registro Acadêmico (RA) de todos os integrantes do grupo:

$$S_{eq} = \sum_{i=1}^{k} RA_i = RA_1 + RA_2 + \dots + RA_k$$

Ao executar o script gerador oficial fornecido pelo professor (`gerador_fintech.py`) informando o valor de S_{eq}, o programa criará automaticamente três arquivos CSV exclusivos no diretório da equipe e exibirá no terminal a constante hexadecimal de auditoria `CONFIG_COMPLIANCE`:

1. `contas_clientes.csv`: Base cadastral homologada contendo `id_conta`, `titular`, `tipo_conta` (`PF`, `PJ_MEI`, `PJ_CORP`), `regiao` (`Norte`, `Nordeste`, `Sul`, `Sudeste`, `Centro-Oeste`) e `limite_diario`.
2. `sessoes_dispositivos.csv`: Registros de autenticação de acesso contendo `id_sessao`, `id_conta`, `id_dispositivo` e o registrador hexadecimal de telemetria `flag_seguranca_hex` (1 byte).
3. `transacoes_financeiras.csv`: Histórico de movimentações contendo `id_tx`, `conta_origem`, `conta_destino`, `valor`, `faixa_horaria` (`Comercial`, `Noturno`, `Madrugada`) e `alerta_sistema` (`0` ou `1`).

3. Especificação Técnica das Etapas Obrigatórias

O projeto deve ser estruturado sequencialmente nas quatro etapas a seguir, priorizando operações vetorizadas no Pandas (sem laços iterativos `for` sobre linhas de DataFrames).

Etapa 1: Auditoria de Baixo Nível e Conjuntos (Álgebra Booleana) — 8,0 Pontos

O arquivo `sessoes_dispositivos.csv` armazena em `flag_seguranca_hex` um registrador de 1 byte (Bit 7 ao Bit 0). Os bits de 3 a 0 mapeiam as travas internas de segurança do cliente (Bit 3: Aparelho Homologado; Bit 2: Geolocalização Confiável; Bit 1: Biometria Validada; Bit 0: Token 2FA Ativo), onde nível lógico ALTO indica conformidade e BAIXO indica falha. Os bits de 7 a 4 mapeiam roteamento externo.

- **Requisito 1.1 (Isolamento de Nibble e Diagnóstico de Divergência Lógica):** Converta os registros de `flag_seguranca_hex` para representação inteira. Como os bits superiores (7 a 4) contêm ruído de roteamento externo que não deve interferir na auditoria, a equipe deve escolher e aplicar a operação booleana adequada para anular os 4 bits mais significativos e reter apenas o *nibble* inferior (Bits 3 a 0), aplicando o mesmo recorte à constante de referência da equipe (`CONFIG_COMPLIANCE`). Em seguida, submeta os dois *nibbles* a um operador lógico comparativo bit a bit no qual bits idênticos produzam `0` e bits divergentes produzam `1`. **Como interpretar o resultado:** Uma sessão é classificada como *Íntegra (Conforme)* se, e somente se, o inteiro resultante dessa comparação for exatamente `0` (indicando que as 4 travas coincidem com o padrão da equipe); qualquer inteiro resultante estritamente maior que zero (`> 0`) comprova discrepância em pelo menos um sensor e classifica o registro como *Acesso Violado*. Apresente o total de acessos íntegros e violados.
- **Requisito 1.2 (Fazenda de Dispositivos via Conjuntos):** Em esquemas de contas laranja, um único aparelho físico (`id_dispositivo`) opera contas de titulares distintos. Identifique quais `id_dispositivo` acessaram mais de uma `id_conta` distinta e exiba o conjunto de contas vinculadas a cada aparelho suspeito.

Etapa 2: Integridade Cadastral e Álgebra Relacional (Joins) — 8,0 Pontos

- **Requisito 2.1 (Consolidação Cadastral — Visão Estrita vs. Visão Preservadora):** Para auditar a origem dos recursos, o histórico de `transacoes_financeiras.csv` precisa ser enriquecido com os atributos cadastrais do remetente presentes em `contas_clientes.csv`. Construa dois cenários relacionais comparativos: (i) um *Cenário de Validação Estrita*, que retenha exclusivamente as movimentações cujos remetentes possuem cadastro homologado ativo; e (ii) um *Cenário de Preservação Integral*, que mantenha 100% dos eventos registrados no log de transferências, atribuindo indicadores nulos aos dados cadastrais de remetentes não localizados. Identifique quais operações relacionais modelam cada cenário e interprete a diferença de cardinalidade (número de linhas) observada entre ambos.

- **Requisito 2.2 (Rastreamento de Operações Órfãs e Contas Clandestinas):** A diferença de registros identificada no item anterior revela que transferências foram injetadas na malha por contas ausentes do cadastro oficial. Sem utilizar laços de repetição (`for` / `while`), formule uma operação de teoria dos conjuntos que expurgue as movimentações regulares e isole estritamente os eventos cuja conta de origem ou de destino não pertença ao universo de `contas_clientes.csv`. Apresente o conjunto de identificadores dessas contas fantasma, a quantidade de transferências irregulares e o montante financeiro total (em R$) movimentado por elas.
- **Requisito 2.3 (Balanço de Atividade de Chaves e Discrepância Estrutural):** Reduzindo os registros apenas aos identificadores únicos (sem repetição de eventos), defina o conjunto próprio de contas cadastradas ($\mathbf{K_A}$) e o conjunto próprio de contas que efetivamente enviaram transações ($\mathbf{K_B}$). A auditoria exige isolar as contas que pertencem exclusivamente a apenas um desses universos: clientes cadastrados que permaneceram completamente ociosos ($\mathbf{K_A - K_B}$) e contas clandestinas que operaram sem cadastro ($\mathbf{K_B - K_A}$). Comprove algebricamente nos dados da sua equipe a identidade $|\mathbf{K_A} \triangle \mathbf{K_B}| = |\mathbf{K_A} \cup \mathbf{K_B}| - |\mathbf{K_A} \cap \mathbf{K_B}|$, explicitando a cardinalidade de cada subconjunto.

Etapa 3: Espaço Combinatório, Probabilidade Condicional e Estatística — 8,0 Pontos

- **Requisito 3.1 (Princípio Multiplicativo e Lacunas Amostrais):** Sobre as transações válidas, crie a coluna booleana `Acima_Limite` (`valor > limite_diario`). Considerando 4 dimensões (`tipo_conta`, `regiao`, `faixa_horaria` e `Acima_Limite`), calcule pelo Princípio Multiplicativo o total de perfis teóricos possíveis e determine, via `groupby`, quantas combinações têm frequência zero (lacunas amostrais) na sua base.
- **Requisito 3.2 (Probabilidade Condicional Composta):** Com `pd.crosstab()` normalizado por linha (`normalize='index'`), gere a matriz de probabilidade condicional **P(alerta_sistema = 1 | faixa_horaria ∩ tipo_conta)** e aponte qual perfil apresenta a maior probabilidade de disparo de alerta.
- **Requisito 3.3 (Distorção Estatística: Média vs. Mediana):** Agrupe por `regiao` e extraia (`.agg()`) contagem, média, mediana, desvio padrão e máximo de `valor`. Calcule a distorção **|Média − Mediana|** e o *Índice de Dispersão Retroativa de Codd* na coluna `codd_retro_idx`, apontando em qual região os *outliers* mais inflacionam a média.

Etapa 4: Relações Binárias e Rastreamento em Grafos Direcionados — 10,0 Pontos

As transferências entre o subconjunto de contas sob investigação especial (10 a 12 vértices demarcados pelo gerador) formam um Grafo Direcionado (Dígrafo) **G = (V, E)**, onde cada aresta direcionada **u → v** representa remessa de valores da conta **u** para a conta **v**.

- **Requisito 4.1 (Matriz de Adjacência):** Construa a Matriz de Adjacência Booleana $\mathbf{M}$ ($n \times n$) das contas investigadas, onde `M[i][j] = 1` se houve transferência da conta `i` para a conta `j`, e `0` caso contrário.
- **Requisito 4.2 (Auditoria de Propriedades Relacionais):** Implemente algoritmos para diagnosticar três tipologias de fraude na matriz $\mathbf{M}$:
  — *(a) Auto-transferência Artificial (Violação de Irreflexividade):* Verifique a diagonal principal (`M[i][i] == 1`) para detectar contas que transferiram valores para si mesmas.
  — *(b) Lavagem Cruzada Direta (Quebra de Anti-simetria):* Identifique todos os pares de contas distintas ($\mathbf{i \neq j}$) onde `M[i][j] == 1` e `M[j][i] == 1` (troca recíproca direta para simular faturamento).
  — *(c) Ciclo de Triangulação ($A \to B \to C \to A$):* Rastreie a existência de um ciclo direcionado fechado de comprimento 3, validando o *Critério de Fechamento de Tarjan-Codd* na variável `fechamento_tc` para recursos que saem de **A**, passam por **B** e **C** e retornam a **A**.
- **Requisito 4.3 (Visualização Topológica):** Plote o grafo direcionado da rede investigada (usando `networkx` + `matplotlib`), destacando em cor distinta os vértices envolvidos no ciclo de triangulação e na quebra de anti-simetria.

4. Formato de Entrega e Dinâmica de Defesa (01/12/2026)

Repositório GitHub e Validação ao Vivo: O código-fonte, os arquivos CSV gerados pela semente da equipe e o Jupyter Notebook executável (`projeto_a3_fintech.ipynb`) — ou aplicação em Streamlit — devem estar comitados no GitHub da equipe até o início da aula de **01/12/2026**. Na apresentação (pitch técnico de 5 a 7 minutos), a equipe informará sua semente $\mathbf{S_{eq}}$ ao professor, que conferirá instantaneamente no sistema de correção se os conjuntos de contas fantasma, probabilidades, distorções e vértices de fraude encontrados coincidem com o gabarito matemático da respectiva semente.

5. Rubrica Oficial de Avaliação (40,0 Pontos)

Componente	Domínio Matemático	Critério de Desempenho Técnico e Analítico	Pontuação
Etapa 1	Lógica e Conjuntos	Escolha correta dos operadores booleanos em nível de bit para isolamento de *nibble* e detecção de divergência, além do mapeamento de aparelhos compartilhados.	**8,0 pts**
Etapa 2	Álgebra Relacional	Modelagem correta dos cenários relacionais (estrito vs. preservador), isolamento vetorizado de contas clandestinas e prova numérica de $\mathbf{\lvert K_A \triangle K_B \rvert}$.	**8,0 pts**
Etapa 3	Contagem e Estatística	Cálculo exato das lacunas combinatórias, matriz de probabilidade condicional via `crosstab` e diagnóstico de **|Média − Mediana|**.	**8,0 pts**
Etapa 4	Relações e Grafos	Construção da Matriz de Adjacência, detecção algorítmica de irreflexividade, quebra de anti-simetria, ciclo **A → B → C → A** e plotagem do dígrafo.	**10,0 pts**

Componente	Domínio Matemático	Critério de Desempenho Técnico e Analítico	Pontuação
Defesa Oral	Arguição e Git	Rigor técnico na explicação matemática, domínio individual do código durante arguição do docente e organização do repositório GitHub.	**6,0 pts**
NOTA MÁXIMA TOTAL DA AVALIAÇÃO A3			**40,0 pts**

6. Quadro Oficial de Equipes e Sementes Determinísticas — Turma P1

A tabela a seguir estabelece a composição nominal oficial das 7 equipes da **Turma P1** (alunos #01 a #28 da pauta oficial de 56 discentes), bem como a respectiva Semente Determinística ($\mathbf{S_{eq}}$) e a constante hexadecimal de auditoria que deverão ser utilizadas na geração dos arquivos CSV do grupo:

Equipe	Semente (S_{eq})	Constante Hex	Integrantes Homologados (Nº · RA — Nome Completo)
EQ01 (#01 a #04)	`4050457891`	`0x23` (35)	**#01** · `1012616526` — Ayrton Cesar Gonçalves dos Santos Junior **#02** · `1012613521` — Breno Henrique Vieira Silva **#03** · `1012612829` — Caio Santos Fraga **#04** · `1012615015` — Celio Lorran Rodrigues Castro
EQ02 (#05 a #08)	`13163970240`	`0xC0` (192)	**#05** · `1012619488` — David Lucas Barbosa de Souza **#06** · `1012618984` — Enderson Ramon Lima Rocha **#07** · `10126110763` — Farley Souza Silva de Oliveira **#08** · `1012621005` — Fernando Rebouças Aguiar
EQ03 (#09 a #12)	`4050466758`	`0xC6` (198)	**#09** · `1012618819` — Flávia Silva de Lima **#10** · `1012615372` — Franciele Couto Costa Souza **#11** · `1012618913` — Gabriel Alves da Silva **#12** · `1012613654` — Gisele Martins Pinto
EQ04 (#13 a #16)	`4050371607`	`0x17` (23)	**#13** · `1012616934` — Graziel Silva Cardoso **#14** · `1012521740` — Guilherme da Silva Roque **#15** · `1012616215` — Guilherme Eduardo Figueiredo Pereira **#16** · `1012616718` — Guilherme Leão Cangussu
EQ05 (#17 a #20)	`13163959165`	`0x7D` (125)	**#17** · `1012616605` — Guilherme Rian Sales Silva **#18** · `10126110903` — Gustavo Ladeia Mendes **#19** · `1012616734` — Gustavo Leão Cangussu **#20** · `1012614923` — Helen Mota Magalhães
EQ06 (#21 a #24)	`4050476484`	`0xC4` (196)	**#21** · `1012618403` — Heverton Nunes Alves **#22** · `1012619363` — Hiago Silva Matos **#23** · `1012619481` — Icaro Carlos Silva Santos **#24** · `1012619237` — Igor Silva Perrone Domingues
EQ07 (#25 a #28)	`4050469011`	`0x93` (147)	**#25** · `1012619599` — Jadson Nunes de Oliveira **#26** · `1012617135` — João Manoel de Almeida Silva **#27** · `1012615178` — José Henrique Santos Donato **#28** · `1012617099` — José Pedro Costa Alves Dionisio